MERN STACK

TECHNICAL INTERVIEW PREPARATION GUIDE

189 carefully organized questions, answers & practical code solutions

CORE AREAS	WHAT YOU WILL PRACTICE
Web & APIs	HTTP, REST, status codes, CORS, pagination, rate limiting
JavaScript	Closures, promises, event loop, ES6+, async/await
React	Hooks, state, effects, routing, forms, performance
Node & Express	Runtime, middleware, routing, controllers, error handling
MongoDB	Mongoose, schema, indexes, aggregation, atomic updates
Security	JWT, bcrypt, XSS, CSRF, validation, authorization
Real projects	Deployment, debugging, testing, concurrency, payments

How to use this guide

1. First understand the answer; do not memorize it word-for-word. 2. Practice explaining each concept aloud in your own words. 3. Run the code snippets in a small test project and adapt them to your codebase. 4. For project questions, explain your own implementation honestly and be ready to discuss trade-offs.

Interview tip: For every feature in your project, be ready to explain the request flow, database design, validation, authentication/authorization, error handling, and one failure case.

Note: The snippets are focused interview examples, not complete production modules. Add validation, logging, tests, configuration, and version-appropriate error handling before using them in a real application.

Contents

1. Web Fundamentals, HTTP & REST APIs — 20 questions

2. JavaScript Essentials — 25 questions

3. React.js — 34 questions

4. Node.js & Express.js — 20 questions

5. MongoDB & Mongoose — 22 questions

6. Authentication, Security & Reliability — 19 questions

7. Practical MERN Architecture, Testing & Deployment — 26 questions

8. Scenario-Based Questions & Code Solutions — 23 questions

Question bank

1. Web Fundamentals, HTTP & REST APIs

Q1. What happens when you enter a URL in a browser?

Answer: The browser checks its cache, resolves the domain through DNS, establishes a connection (TCP, and usually TLS for HTTPS), sends an HTTP request, receives a response, and parses HTML/CSS/JavaScript. The browser builds the DOM and CSSOM, performs layout and painting, then executes JavaScript. A React app typically downloads a JS bundle and renders the UI.

Q2. What is the difference between HTTP and HTTPS?

Answer: HTTP transfers data without transport encryption. HTTPS is HTTP over TLS: it encrypts traffic, helps detect tampering, and authenticates the server using a certificate. Use HTTPS for login, cookies, APIs, and production applications.

Q3. What is an HTTP request and response?

Answer: A request contains a method, URL, headers, and optionally a body. A response contains a status code, headers, and optionally a body. Example: a browser sends GET /api/products; the server responds with 200 OK and JSON product data.

Q4. Explain common HTTP methods.

Answer: GET retrieves data; POST creates a resource or triggers an operation; PUT replaces a resource; PATCH partially updates it; DELETE removes it. GET should be safe and typically idempotent. PUT and DELETE are defined as idempotent; POST generally is not.

Q5. What is REST?

Answer: REST (Representational State Transfer) is an architectural style for networked applications. It models data as resources identified by URLs and uses standard HTTP methods, stateless requests, representations such as JSON, and meaningful status codes. REST is a style, not a library or protocol.

Q6. What makes an API RESTful?

Answer: Resource-oriented URLs, correct HTTP methods, stateless requests, standard status codes, consistent representations, and predictable resource relationships. For example, GET /api/orders/42 reads order 42 and PATCH /api/orders/42 updates selected fields.

Q7. What is the difference between REST and SOAP?

Answer: REST is an architectural style commonly used with JSON over HTTP and is usually lightweight. SOAP is a formal messaging protocol with XML envelopes and strict standards for contracts, security, and transactions. They solve overlapping problems but are not interchangeable terms.

Q8. What is JSON?

Answer: JavaScript Object Notation is a text format for structured data. It supports objects, arrays, strings, numbers, booleans, and null. JSON is language-independent and commonly used as the request/response format between React and Express.

Q9. What are HTTP status-code categories?

Answer: 1xx is informational, 2xx success, 3xx redirection, 4xx client-side error, and 5xx server-side error. Common examples: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Content, and 500 Internal Server Error.

Q10. Differentiate 401 and 403.

Answer: 401 means the request lacks valid authentication credentials or the credentials are invalid. 403 means the server understood the request but refuses authorization. A logged-out user may receive 401; an authenticated student trying to access a mentor-only route may receive 403.

Q11. What is the difference between PUT and PATCH?

Answer: PUT generally replaces the complete representation of a resource, while PATCH applies a partial modification. If a user profile has name, email, and phone, PATCH can update only phone; a PUT request normally sends the complete intended profile representation.

Q12. What is CORS?

Answer: Cross-Origin Resource Sharing is a browser-enforced mechanism that lets a server declare which origins may read its responses. An origin is the scheme, host, and port. Configure Express with an explicit frontend origin and credentials when using cookie-based authentication; CORS is not authentication.

Q13. What is a preflight request?

Answer: For certain cross-origin requests, the browser first sends OPTIONS with headers such as Origin and Access-Control-Request-Method. The server responds with allowed origins, methods, and headers. The browser sends the actual request only if the preflight succeeds.

Q14. What is the difference between cookies, localStorage, and sessionStorage?

Answer: Cookies are automatically attached to matching requests and can be HttpOnly, Secure, and SameSite. localStorage persists until cleared and is accessible to JavaScript. sessionStorage is scoped to a tab session. Sensitive session tokens are often safer in appropriately configured HttpOnly cookies than in JavaScript-readable storage.

Q15. What is an API endpoint?

Answer: An endpoint is a specific API address and method combination, such as GET /api/products or POST /api/orders. The method plus path identifies the operation; the endpoint should have a clear request and response contract.

Q16. What is API versioning and why use it?

Answer: Versioning allows an API to evolve without unexpectedly breaking existing clients. Common approaches include /api/v1/products or version headers. Introduce a new version for breaking changes and communicate deprecation timelines.

Q17. What is pagination?

Answer: Pagination divides a large result set into smaller responses. Offset pagination uses page and limit; cursor pagination uses a stable cursor such as _id or createdAt. Cursor pagination is often more reliable for frequently changing datasets.

Q18. What is rate limiting?

Answer: Rate limiting restricts how many requests a client can make in a time window. It helps reduce abuse, brute-force attempts, and accidental overload. Use a shared store such as Redis in multi-instance deployments; in-memory counters are not shared across server instances.

Q19. What is idempotency in APIs?

Answer: An operation is idempotent if repeating the same request has the same intended effect as performing it once. This matters when clients retry after network failures. For payments or bookings, an idempotency key can prevent duplicate processing.

Q20. How would you design an API to reserve a unique item for the first buyer?

Answer: Do not rely on disabling a button in React. Perform an atomic database update that changes status only if it is still available, for example findOneAndUpdate({_id, status:'available'}, {$set:{status:'reserved', reservedBy:userId}}). Only one concurrent request can match the available state; return 409 Conflict to the others.

2. JavaScript Essentials

Q21. What are the differences between var, let, and const?

Answer: var is function-scoped and can be redeclared; let and const are block-scoped. let can be reassigned, while const cannot be reassigned (but an object held by const can still be mutated). Prefer const by default and let when reassignment is required.

Q22. What is hoisting?

Answer: JavaScript processes declarations before executing code. Function declarations are available before their textual position. var declarations are hoisted and initialized to undefined. let and const are hoisted but remain in the temporal dead zone until their declaration is evaluated.

Q23. What is the difference between == and ===?

Answer: == performs type coercion before comparison in many cases; === compares without coercing types. Prefer === to avoid surprising conversions, such as '5' == 5 being true while '5' === 5 is false.

Q24. What are primitive and reference types?

Answer: Primitive values include string, number, bigint, boolean, undefined, symbol, and null. Objects, arrays, and functions are reference values. Assigning an object variable copies the reference, not a deep copy of the object.

Q25. What is a closure?

Answer: A closure is a function bundled with access to variables from its lexical scope, even after the outer function has returned. Closures are useful for private state, callbacks, and React hooks.

Q26. Explain the this keyword.

Answer: The value of this depends on how a function is called. In a method call it commonly refers to the receiver object; in a regular function it depends on strict mode and call site; arrow functions capture this lexically. Avoid assuming this is the object where a function was written.

Q27. What is the difference between an arrow function and a regular function?

Answer: Arrow functions have lexical this and cannot be used as constructors. Regular functions have their own this based on invocation and can be constructors. Both can be used as callbacks, but choose based on the required behavior.

Q28. What are map, filter, and reduce?

Answer: map transforms every item and returns a new array; filter returns items satisfying a condition; reduce accumulates an array into one result. They are useful for rendering and transforming API data.

Q29. What is destructuring?

Answer: Destructuring extracts values from arrays or properties from objects into variables. Example: const { name, email } = user. It improves readability when working with API responses and component props.

Q30. What is the spread operator?

Answer: Spread expands iterable values or enumerable object properties. [...items] creates a shallow array copy; {...user, city:'Pune'} creates a shallow object copy with an updated city. Nested objects remain shared references.

Q31. What is the rest parameter?

Answer: The rest parameter collects remaining function arguments into an array, e.g. function sum(...numbers). It differs from spread by collecting rather than expanding.

Q32. What is a callback function?

Answer: A callback is a function passed to another function to be called later. Event handlers, array methods, and asynchronous APIs commonly use callbacks. Excessive nested callbacks can create callback hell.

Q33. What is a Promise?

Answer: A Promise represents the eventual result of an asynchronous operation. It can be pending, fulfilled, or rejected. Use then/catch/finally or async/await to consume it.

Q34. Explain async and await.

Answer: async makes a function return a Promise. await pauses that async function until the Promise settles, without blocking the JavaScript event loop. Use try/catch for rejected operations.

Q35. What is the event loop?

Answer: JavaScript executes synchronous code on a call stack. Asynchronous work is handled by the host environment; callbacks are queued. The event loop schedules queued work when the stack is clear. Promise reactions (microtasks) are generally processed before the next task such as a timer callback.

Q36. What is the difference between synchronous and asynchronous code?

Answer: Synchronous code runs in sequence and blocks the current execution until it finishes. Asynchronous code allows the program to continue while an operation is pending, then handles its result later. Network calls and file operations are common asynchronous tasks.

Q37. What is optional chaining?

Answer: Optional chaining (?.) safely accesses a property or calls a function when the preceding value may be null or undefined. Example: user?.address?.city. It returns undefined rather than throwing for a nullish intermediate value.

Q38. What is nullish coalescing?

Answer: The ?? operator uses the right-hand value only when the left-hand value is null or undefined. Unlike ||, it preserves valid falsy values such as 0 and an empty string.

Q39. What is the difference between shallow copy and deep copy?

Answer: A shallow copy duplicates only the top-level structure; nested objects remain shared. A deep copy recursively duplicates nested data. structuredClone supports many common data types, but check compatibility and avoid using deep cloning as a substitute for thoughtful state updates.

Q40. What is error handling in JavaScript?

Answer: Use try/catch for synchronous errors and awaited Promise rejections. Handle expected failures explicitly, validate inputs, and avoid swallowing errors. At API boundaries, log useful diagnostic details on the server but return safe messages to clients.

Q41. What is the difference between Promise.all and Promise.allSettled?

Answer: Promise.all fulfills when every input fulfills and rejects as soon as one rejects. Promise.allSettled waits for all inputs and reports each fulfilled/rejected result. Use all when every result is required; use allSettled when independent tasks should all be reported.

Q42. What is debouncing?

Answer: Debouncing delays a function until a period of inactivity has passed. It is useful for search inputs so an API request is not sent on every keystroke. The timer must be cleaned up when a React component unmounts or the query changes.

Q43. What is throttling?

Answer: Throttling limits a function to at most one execution per interval. It is useful for scroll, resize, or repeated interaction handlers. Debouncing waits for inactivity; throttling enforces a maximum frequency.

Q44. What is the difference between for...in and for...of?

Answer: for...in iterates enumerable property keys, often used for object keys. for...of iterates values from an iterable such as an array, string, or Map. Use the loop that matches the data structure.

Q45. What are ES modules?

Answer: ES modules use import and export to share code between files. They support static analysis and explicit dependencies. Node.js supports ESM when configured appropriately, while CommonJS uses require and module.exports.

3. React.js

Q46. What is React?

Answer: React is a JavaScript library for building user interfaces from reusable components. It describes UI as a function of state and props; when relevant data changes, React updates the rendered interface.

Q47. What is a component?

Answer: A component is a reusable unit of UI, usually a function that accepts props and returns JSX. Components help divide a large interface into smaller, testable pieces such as Navbar, ProductCard, Cart, and Dashboard.

Q48. What is JSX?

Answer: JSX is a syntax extension that lets developers write markup-like expressions in JavaScript. Build tools transform JSX into JavaScript calls that create React elements. JSX is not HTML, even though it looks similar.

Q49. What is the Virtual DOM?

Answer: React elements describe the desired UI. React reconciles the new element tree with the previous one and commits necessary changes to the real DOM. 'Virtual DOM' is a useful mental model, not a guarantee that every update is faster than direct DOM work.

Q50. What are props?

Answer: Props are read-only inputs passed from a parent component to a child. They allow components to be configured and reused. A child should not mutate its props.

Q51. What is state in React?

Answer: State is component-managed data that can change over time. Updating state schedules a render. State should be treated as immutable; create new arrays or objects rather than mutating existing state directly.

Q52. What is the difference between props and state?

Answer: Props are passed into a component by its parent; state is managed by the component (or a state owner) itself. Both influence rendering, but state updates are performed through the relevant setter or state-management mechanism.

Q53. What is the useState hook?

Answer: useState adds local state to a function component. It returns the current value and a setter. The setter schedules a render; when the next value depends on the previous value, use the functional form: setCount(prev => prev + 1).

Q54. What is useEffect?

Answer: useEffect synchronizes a component with an external system, such as fetching data, subscribing to events, or updating a browser API. Its setup runs after commit, and its cleanup runs before a re-run and on unmount. Do not use it for every derived value.

Q55. Explain the useEffect dependency array.

Answer: No dependency array means the effect runs after every commit. An empty array means it runs after the initial mount (and cleanup on unmount, with development Strict Mode behavior to expose unsafe effects). A dependency array reruns the effect when a dependency changes.

Q56. What is the cleanup function in useEffect?

Answer: An effect may return a cleanup function. Use it to unsubscribe, clear timers, abort requests, or remove event listeners. Cleanup prevents stale subscriptions and memory leaks.

Q57. What are React hooks?

Answer: Hooks are functions such as useState, useEffect, useMemo, useRef, and custom hooks that let function components use React features. Call hooks only at the top level of React components or custom hooks, not inside loops, conditions, or nested functions.

Q58. What is a custom hook?

Answer: A custom hook is a reusable function whose name starts with use and which can call other hooks. It extracts reusable stateful logic, not necessarily UI. Example: useAuth can expose the current user and authentication actions.

Q59. What is useContext?

Answer: useContext reads a value from the nearest matching Context provider. It helps share values such as theme, authenticated user, or locale without passing props through every intermediate component. Frequent broad context updates can cause many consumers to rerender.

Q60. What is useRef?

Answer: useRef returns a stable object whose .current property persists between renders. Updating it does not trigger a render. It is commonly used for DOM references, timer IDs, and mutable values that do not directly determine the UI.

Q61. What is useMemo?

Answer: useMemo caches a calculated value between renders until dependencies change. Use it for measured expensive calculations or stable derived values when needed; do not add it everywhere because memoization has overhead.

Q62. What is useCallback?

Answer: useCallback caches a function reference until dependencies change. It can help when passing callbacks to memoized children or when a stable callback is required by another hook. It does not make the function's execution intrinsically faster.

Q63. What is React.memo?

Answer: React.memo can skip rerendering a function component when its props are shallowly equal to the previous props. It is a performance optimization, not a correctness mechanism. New object/function props can defeat the shallow comparison.

Q64. What is prop drilling?

Answer: Prop drilling is passing data through several intermediate components that do not need it, just to reach a deeply nested child. Context, composition, or a state-management library may help when the pattern becomes difficult to maintain.

Q65. What are keys in React lists?

Answer: Keys give list elements stable identities so React can match items between renders. Use stable unique IDs from data, not array indexes when items can be inserted, removed, or reordered. Keys are not passed as normal component props.

Q66. What is conditional rendering?

Answer: Conditional rendering displays different UI based on state or props, using if statements, ternaries, or logical operators. Always consider loading, empty, success, and error states for API-driven screens.

Q67. What are controlled and uncontrolled components?

Answer: A controlled input gets its value from React state and updates through an onChange handler. An uncontrolled input stores its value in the DOM and can be read through a ref. Controlled inputs are useful for validation and predictable form behavior.

Q68. How do you handle forms in React?

Answer: Keep form values in state (or use a form library), validate on the client for user experience, and always validate again on the server. Handle submission asynchronously, show loading/error states, and prevent duplicate submissions.

Q69. What is React Router?

Answer: React Router maps URLs to UI components in a single-page application. It supports nested routes, route parameters, navigation, and protected-route patterns. Authorization must still be enforced by the backend.

Q70. What is lazy loading in React?

Answer: React.lazy can load a component on demand, and Suspense can show a fallback while it loads. Route-level code splitting reduces the initial JavaScript bundle for large applications.

Q71. What is reconciliation?

Answer: Reconciliation is React's process of comparing the new element description with the previous one to determine what should change. Component type, position, and keys influence whether state is preserved or reset.

Q72. What is StrictMode?

Answer: StrictMode is a development-only tool that helps detect unsafe patterns. In current React development behavior, it may intentionally rerun certain render/effect setup cycles to expose missing cleanup. It does not add the same extra behavior to production builds.

Q73. How do you avoid unnecessary rerenders?

Answer: Keep state close to where it is used, avoid needless state duplication, split components, use stable keys, and measure with React DevTools Profiler. Consider memo, useMemo, and useCallback only when profiling or a clear reference-stability need justifies them.

Q74. How do you fetch data in React?

Answer: Use fetch or a client such as Axios inside an effect or a data-fetching library. Track loading, data, and error states; cancel or ignore stale requests; and consider caching, retries, and invalidation for larger applications.

Q75. What is the difference between client-side and server-side rendering?

Answer: Client-side rendering builds much of the UI in the browser after JavaScript loads. Server-side rendering generates HTML on the server for a request and then hydrates it on the client. SSR can improve initial content delivery and SEO but adds server complexity.

Q76. What is hydration?

Answer: Hydration attaches React's event handling and client behavior to HTML that was rendered on the server. The server and client output should match; mismatches can cause warnings or replacement rendering.

Q77. How would you implement protected routes?

Answer: Check authentication state in the frontend and redirect unauthenticated users for a good user experience. Also protect every sensitive Express endpoint with authentication and role/ownership authorization; frontend route guards alone are not security.

Q78. How do you manage global state?

Answer: Use Context for relatively simple shared state, or a dedicated library such as Redux Toolkit or Zustand when state transitions, debugging, middleware, or scale justify it. Server state may be better managed with TanStack Query or a similar data-fetching library.

Q79. What is an error boundary?

Answer: An error boundary is a React component that catches rendering errors in its descendant tree and displays fallback UI. It does not catch every event-handler error or every asynchronous error; those need their own handling.

4. Node.js & Express.js

Q80. What is Node.js?

Answer: Node.js is a JavaScript runtime built on the V8 engine. It provides APIs for networking, files, processes, and other server-side tasks. It is commonly used to build APIs and real-time services.

Q81. Is Node.js single-threaded?

Answer: JavaScript execution in a typical Node.js event loop runs on one main thread, but Node.js and its libraries use operating-system facilities and a worker pool for certain work. Worker Threads can run CPU-intensive JavaScript in parallel. Avoid saying Node.js can never use multiple threads.

Q82. What is the event-driven, non-blocking I/O model?

Answer: Node.js starts I/O operations and registers callbacks or Promise continuations rather than waiting synchronously for each operation to finish. This allows one process to handle many concurrent I/O-bound requests efficiently.

Q83. What is npm?

Answer: npm is the package manager commonly used with Node.js. It installs dependencies, manages package metadata and lockfiles, and runs scripts defined in package.json. npm is also the name of the package registry.

Q84. What is package.json?

Answer: package.json describes a Node project, including its name, scripts, dependencies, development dependencies, and configuration. It is not the exact installed dependency tree; package-lock.json records a resolved dependency graph for reproducible installs.

Q85. What is the difference between dependencies and devDependencies?

Answer: dependencies are needed by the application at runtime. devDependencies are mainly used for development, testing, linting, and building. Production deployment should install the dependencies required by the deployed process.

Q86. What is Express.js?

Answer: Express is a minimal web framework for Node.js. It provides routing, middleware integration, and request/response helpers, making it convenient to build REST APIs.

Q87. What is middleware in Express?

Answer: Middleware is a function that can inspect or modify req and res, end the response, or call next(). Middleware is used for logging, authentication, validation, parsing, CORS, and error handling. Order matters.

Q88. Explain the Express request-response cycle.

Answer: A request enters the middleware stack in registration order. Middleware may modify the request, reject it, or pass control with next(). A matching route handler performs the operation and sends a response. Errors are passed to error-handling middleware.

Q89. What is the difference between app.use and app.get?

Answer: app.use registers middleware, optionally mounted at a path, and can apply to multiple HTTP methods. app.get registers a route handler for GET requests at a path.

Q90. What is express.json()?

Answer: express.json() is built-in middleware that parses incoming JSON request bodies and populates req.body when the content type matches. It does not validate the shape or trustworthiness of the input.

Q91. What is express.Router()?

Answer: express.Router() creates a modular router that groups related routes. For example, product routes can live in productRoute.js and be mounted at /api/products in the main application.

Q92. How do you handle errors in Express?

Answer: Use centralized error middleware with the four parameters (err, req, res, next). Forward errors to it, return consistent status codes and safe messages, and log diagnostic details on the server. In async handlers, ensure rejected Promises are forwarded correctly for your Express version.

Q93. What is the purpose of next()?

Answer: next() passes control to the next middleware or matching route handler. next(err) forwards an error to error middleware. Calling next() after sending a response can cause duplicate-response errors.

Q94. What is environment configuration?

Answer: Configuration such as PORT, database URI, JWT secret, and payment keys should come from environment variables or a secret manager. Keep real secrets out of source control; provide a safe example .env.example without actual credentials.

Q95. What is the difference between process.env and dotenv?

Answer: process.env exposes environment variables to the Node process. dotenv is a package that loads variables from a local .env file into process.env, commonly for development. Production platforms should inject secrets through their configuration system.

Q96. What is nodemon?

Answer: nodemon watches project files and restarts the Node process when files change. It is a development convenience, not normally a production process manager.

Q97. How do you structure an Express project?

Answer: A practical structure separates routes, controllers, services, models, middleware, validators, config, and utilities. Routes map URLs, controllers handle HTTP concerns, services contain business logic, and models handle persistence.

Q98. What is the difference between authentication and authorization?

Answer: Authentication establishes who the user is. Authorization determines what that authenticated user may do. A valid login does not automatically grant access to every resource.

Q99. How would you implement role-based access control?

Answer: Authenticate the request, load trusted role information from the verified session/token or database, and apply middleware that checks the required role. Also check resource ownership where needed; a role check alone may not prevent one user accessing another user's record.

5. MongoDB & Mongoose

Q100. What is MongoDB?

Answer: MongoDB is a document-oriented NoSQL database. It stores BSON documents in collections and supports flexible document structures, indexes, aggregation, and transactions.

Q101. What is a document and a collection?

Answer: A document is a BSON record made of fields and values, similar conceptually to a JSON object. A collection groups documents, similar conceptually to a relational table, though documents need not all have identical fields.

Q102. What is BSON?

Answer: BSON is MongoDB's binary-encoded serialization format. It extends JSON-like data with types such as ObjectId, dates, and binary data, and is used internally for storage and network communication.

Q103. What is the difference between SQL and NoSQL databases?

Answer: Relational databases organize data into tables with defined relationships and commonly use SQL. Document databases store flexible documents and often model data around access patterns. Neither is universally better; choose based on consistency, query, relationship, and scaling requirements.

Q104. What is an ObjectId?

Answer: ObjectId is a BSON identifier type commonly used as MongoDB's default _id. It is designed to be highly likely unique and includes a timestamp component, but it should not be treated as a secret or authorization token.

Q105. What is Mongoose?

Answer: Mongoose is an ODM (Object Data Modeling) library for Node.js and MongoDB. It provides schemas, models, validation, middleware, and query helpers.

Q106. What is a Mongoose schema?

Answer: A schema defines the expected structure, types, validation rules, defaults, and options for documents in a Mongoose model. MongoDB itself is flexible; Mongoose schema rules are enforced by the application layer.

Q107. What is a Mongoose model?

Answer: A model is a compiled constructor based on a schema. It provides methods to create, query, update, and delete documents in the associated collection.

Q108. What is the difference between find, findOne, and findById?

Answer: find returns an array of matching documents. findOne returns the first matching document or null. findById is a convenience method that queries by _id.

Q109. What is the difference between save and findByIdAndUpdate?

Answer: save operates on a document instance and runs document validation and save middleware. findByIdAndUpdate performs a query update and has different middleware/validation behavior; use options such as runValidators where appropriate and understand the query middleware lifecycle.

Q110. What is populate()?

Answer: populate() replaces referenced ObjectId fields with documents from another collection. It is convenient for related data, but excessive population can cause extra database work; select only the fields needed.

Q111. What is embedding versus referencing?

Answer: Embedding stores related data inside one document and is useful when data is read/updated together and remains bounded. Referencing stores IDs to separate documents and is useful for shared, independently changing, or large related data. Choose based on access patterns and document growth.

Q112. What are indexes in MongoDB?

Answer: Indexes improve query performance by allowing MongoDB to locate documents without scanning the entire collection. They consume storage and add write overhead. Index fields and order should match real query and sort patterns.

Q113. What is a compound index?

Answer: A compound index includes multiple fields in a defined order. Field order matters, and the index can support queries on the leading prefix. Design it based on equality, sort, and range patterns.

Q114. What is the aggregation pipeline?

Answer: Aggregation processes documents through stages such as $match, $group, $project, $sort, and $lookup. It is useful for reports, totals, analytics, and transformations that would otherwise require substantial application-side processing.

Q115. What is the difference between deleteOne and findOneAndDelete?

Answer: deleteOne removes a matching document and returns deletion metadata. findOneAndDelete finds and removes one document and returns the deleted document (or null), which can be useful when the application needs the removed record.

Q116. What is schema validation?

Answer: Schema validation checks that stored or incoming data meets expected rules, such as required fields, types, ranges, and enums. Use Mongoose validation and request validation, and consider MongoDB collection validation for database-level protection.

Q117. What is a MongoDB transaction?

Answer: A transaction groups multiple database operations so they commit or abort together, subject to MongoDB deployment requirements. Use transactions when multiple writes must remain consistent, but prefer atomic single-document operations when they solve the problem more simply.

Q118. What is atomicity in MongoDB?

Answer: An operation on a single document is atomic. This is powerful for inventory or reservation updates: include the expected current state in the update filter so competing requests cannot both change the same item from available to reserved.

Q119. How do you prevent duplicate records?

Answer: Use a unique index for the actual uniqueness rule, such as unique email or a compound unique key. Application-side 'check then insert' alone is race-prone because two requests can pass the check concurrently. Handle duplicate-key errors gracefully.

Q120. How do you connect MongoDB to Express?

Answer: Use the MongoDB driver or Mongoose, keep the URI in an environment variable, connect during application startup, and handle connection failures. Avoid opening a new database connection for every HTTP request.

Q121. What is lean() in Mongoose?

Answer: lean() returns plain JavaScript objects instead of full Mongoose documents. It can reduce overhead for read-only queries, but those results do not have document methods, getters, setters, or normal document change tracking.

6. Authentication, Security & Reliability

Q122. What is JWT?

Answer: JSON Web Token is a compact token format containing claims, commonly signed so a server can verify integrity. A typical access token contains a user identifier and expiry. A signed JWT is not automatically encrypted; do not put secrets in its payload.

Q123. Explain the JWT authentication flow.

Answer: The user submits credentials over HTTPS. The server verifies the password hash and issues a short-lived signed access token (and optionally a refresh token). The client sends the access token on protected requests; middleware verifies signature, expiry, and relevant claims before authorizing access.

Q124. What is the difference between access and refresh tokens?

Answer: Access tokens are short-lived credentials used to call APIs. Refresh tokens live longer and are used to obtain new access tokens. Refresh tokens should be protected, rotated where appropriate, and revocable; do not treat them as ordinary access tokens.

Q125. Where should JWTs be stored?

Answer: Storage depends on the threat model. HttpOnly, Secure, SameSite cookies reduce direct JavaScript access but require CSRF-aware design. localStorage is convenient but accessible to injected JavaScript in an XSS attack. Use HTTPS, short expiry, CSP, careful cookie settings, and a considered CSRF strategy.

Q126. What is password hashing and why use bcrypt?

Answer: Hashing transforms a password into a one-way representation. bcrypt is a password-hashing function designed to be computationally expensive and supports a configurable work factor. Never store plaintext passwords or use fast general-purpose hashes such as plain SHA-256 for passwords.

Q127. What is salting?

Answer: A salt is a unique random value incorporated into password hashing. It prevents identical passwords from producing identical stored hashes and makes precomputed rainbow-table attacks less useful. bcrypt and modern password-hashing libraries manage salts for you.

Q128. What is XSS?

Answer: Cross-Site Scripting occurs when attacker-controlled content is executed as script in another user's browser. Prevent it by escaping output, avoiding unsafe HTML injection, sanitizing permitted rich text, using a Content Security Policy, and protecting tokens.

Q129. What is CSRF?

Answer: Cross-Site Request Forgery tricks a browser into sending an authenticated request to a site where the user is logged in. Mitigations include SameSite cookies, CSRF tokens for relevant cookie-based flows, origin checks, and avoiding state-changing GET requests.

Q130. What is SQL/NoSQL injection?

Answer: Injection happens when untrusted input changes the meaning of a database query. Use parameterized queries or safe ODM query construction, validate input types, avoid passing arbitrary client objects directly as filters, and restrict database privileges.

Q131. What is Helmet in Express?

Answer: Helmet is middleware that sets several HTTP security headers. It can help reduce certain browser-side risks, but it does not replace input validation, authorization, secure cookies, or a Content Security Policy tailored to the app.

Q132. Why is input validation important?

Answer: Client-side validation improves usability but can be bypassed. Validate every untrusted value on the server: types, length, formats, allowed fields, IDs, and business rules. Reject unexpected fields where practical and avoid trusting req.body.

Q133. What is authorization by resource ownership?

Answer: It checks whether the authenticated user is allowed to access a specific record, not merely whether they have a role. For example, a user should only update an order if the order belongs to them or they have an explicitly authorized staff role.

Q134. How do you secure file uploads?

Answer: Restrict size and allowed file types, inspect actual content rather than trusting the extension, generate safe filenames, store outside executable web roots or in object storage, scan where appropriate, and enforce authorization. Never trust a client-provided path.

Q135. What is a secure way to store API secrets?

Answer: Use environment variables injected by the deployment platform or a dedicated secrets manager. Never commit .env files, print secrets in logs, or expose server-only keys through frontend build variables.

Q136. How do you handle brute-force login attempts?

Answer: Apply rate limits, progressive delays or temporary lockouts with care, monitor suspicious activity, use strong password hashing, and consider MFA. Avoid account-enumeration messages that reveal whether an email exists.

Q137. What is the principle of least privilege?

Answer: Give each user, service, and database account only the permissions required for its task. For example, an API's database user should not have unnecessary administrative privileges.

Q138. How do you prevent sensitive data leakage in API responses?

Answer: Return explicit allowlisted fields, never serialize password hashes or internal secrets, use DTOs or projection, and avoid sending stack traces or raw database errors to clients. Review logs and error messages too.

Q139. What is a security issue with trusting a role sent by the frontend?

Answer: The client can modify its own JavaScript or HTTP request. The backend must derive identity and permissions from verified credentials and trusted database state, not from a role field supplied in req.body or localStorage.

Q140. What is a race condition?

Answer: A race condition occurs when concurrent operations interact in an order that produces an incorrect result. Examples include two users reserving the same item or two requests decrementing stock. Use atomic database operations, unique constraints, transactions where needed, and idempotency.

7. Practical MERN Architecture, Testing & Deployment

Q141. Explain the complete MERN architecture.

Answer: React renders the frontend and sends HTTP requests. Express defines API routes and middleware on Node.js. Controllers/services implement business rules. Mongoose communicates with MongoDB. The API returns JSON, and React updates the UI. Authentication, validation, logging, and error handling cross these layers.

Q142. What is MVC?

Answer: MVC separates an application into Model (data and persistence), View (presentation), and Controller (request handling and coordination). In a MERN API, React commonly owns the UI, while Express controllers coordinate models and services. A service layer can keep business logic out of controllers.

Q143. How should frontend and backend communicate?

Answer: Define stable endpoints and JSON contracts. The frontend sends HTTP requests using fetch/Axios, handles loading/success/error states, and the backend validates requests, authorizes the user, executes business logic, and returns appropriate status codes and data.

Q144. What is Axios and how is it different from fetch?

Answer: Both make HTTP requests. fetch is built into modern browsers and does not reject merely because the server returns a 4xx/5xx status; check response.ok. Axios provides convenient defaults, interceptors, and automatic JSON handling. Either is suitable when used consistently.

Q145. What are interceptors in Axios?

Answer: Request interceptors can add headers or attach tokens; response interceptors can centralize error handling or refresh logic. Avoid infinite refresh loops, concurrent refresh races, and blindly retrying non-idempotent operations.

Q146. How do you handle loading and error states?

Answer: Model the request lifecycle explicitly: idle, loading, success, and error (or equivalent state). Disable duplicate submissions when appropriate, show useful feedback, and distinguish empty results from failed requests.

Q147. How do you organize environment variables in a MERN app?

Answer: Keep frontend and backend configuration separate. Frontend variables are generally bundled into public client code and must never contain secrets. Backend secrets such as database URIs and signing keys stay server-side.

Q148. How do you deploy a MERN application?

Answer: Build and deploy the React frontend to a static hosting service or suitable app platform; deploy the Node/Express API to a server platform; provision MongoDB; configure environment variables, CORS, HTTPS, health checks, and allowed origins; then test production flows and logs.

Q149. What is the purpose of a build step in React?

Answer: The build step transforms source code, bundles modules, optimizes assets, and produces production-ready static files. It may minify code, split chunks, and process CSS and assets.

Q150. What is CI/CD?

Answer: Continuous Integration automatically builds and tests changes when code is pushed. Continuous Delivery/Deployment automates preparing or releasing validated changes. A pipeline may run linting, tests, build checks, and deployment.

Q151. What is unit testing?

Answer: Unit testing checks a small unit, such as a utility function or isolated service, in controlled conditions. Tests should cover expected behavior, edge cases, and failures.

Q152. What is integration testing?

Answer: Integration testing checks whether multiple parts work together, such as an Express route with middleware and a test database. It catches issues that isolated unit tests may miss.

Q153. What is end-to-end testing?

Answer: E2E testing exercises a complete user journey through the application, often in a real browser: sign in, browse products, add to cart, and place an order. It gives confidence in integrated behavior but can be slower and more brittle.

Q154. What is Postman used for?

Answer: Postman helps send API requests, inspect responses, organize collections, configure environments, and automate API checks. It is useful for testing endpoints independently of the React frontend.

Q155. How do you debug a MERN application?

Answer: Reproduce the issue, inspect browser console and Network tab, check request payload/status/response, inspect server logs, isolate the failing layer, verify database state, and add a regression test. Avoid logging passwords, tokens, or personal data.

Q156. How do you improve API performance?

Answer: Measure first. Add appropriate indexes, limit returned fields, paginate, avoid N+1 queries and excessive populate calls, cache suitable read-heavy data, compress responses when beneficial, and move CPU-heavy work off the event loop.

Q157. What is caching?

Answer: Caching stores reusable results so repeated requests can be served faster. Options include browser caching, CDN caching, in-process caches, and Redis. Define expiry and invalidation carefully; never share user-specific data across users accidentally.

Q158. What is Redis used for in a MERN application?

Answer: Redis is an in-memory data store often used for caching, rate-limit counters, sessions, queues, and ephemeral coordination. It complements rather than replaces MongoDB for many applications.

Q159. What is WebSocket?

Answer: WebSocket provides a persistent, bidirectional connection between client and server, useful for chat, live notifications, and real-time status. It differs from ordinary request-response HTTP, though WebSocket begins with an HTTP handshake.

Q160. When would you use polling instead of WebSockets?

Answer: Polling is simpler when updates are infrequent and slight delay is acceptable. Long polling or short polling may be sufficient for basic status refresh. WebSockets are useful for frequent, low-latency, bidirectional updates but add connection and scaling complexity.

Q161. What is a background job queue?

Answer: A queue moves slow or retryable work out of the request-response path, such as sending emails, generating reports, or processing images. A worker consumes jobs asynchronously; jobs should be idempotent and have retry/backoff and failure handling.

Q162. What is logging and monitoring?

Answer: Logging records structured events and errors with useful context. Monitoring tracks metrics such as latency, error rate, CPU, memory, and database health. Use request IDs and redact secrets and sensitive personal information.

Q163. What is graceful shutdown in Node.js?

Answer: On termination, stop accepting new requests, allow in-flight requests to finish within a deadline, close database and queue connections, and exit. This helps deployments restart without abruptly dropping work.

Q164. How do you handle file/image storage in a MERN project?

Answer: Store files in object storage or a managed media service rather than relying on a server's temporary filesystem. Save the file URL and metadata in MongoDB, validate uploads, secure access, and configure transformations/CDN delivery where useful.

Q165. What is the difference between monolith and microservices?

Answer: A monolith deploys the application as one unit and is often simpler for small teams. Microservices split capabilities into independently deployable services but add network, data consistency, observability, and operational complexity. Start with a modular monolith unless independent scaling or ownership justifies services.

Q166. What is a health-check endpoint?

Answer: A health endpoint reports whether the service is alive and/or ready to accept traffic. Liveness checks should be lightweight; readiness can verify critical dependencies such as database connectivity. Do not expose sensitive infrastructure details publicly.

8. Scenario-Based Questions & Code Solutions

Q167. How would you implement a basic GET endpoint for products?

Answer: Define a route, validate pagination/filter parameters, query the Product model, and return a consistent JSON response. Keep database logic in a controller or service and handle errors centrally.

Example solution

```
// routes/productRoutes.js
router.get("/", productController.getProducts);

// controllers/productController.js
export async function getProducts(req, res, next) {
  try {
    const page = Math.max(1, Number(req.query.page) || 1);
    const limit = Math.min(50, Math.max(1, Number(req.query.limit) || 10));
    const products = await Product.find({})
      .select("name price image")
      .skip((page - 1) * limit)
      .limit(limit)
      .lean();
    res.status(200).json({ data: products, page, limit });
  } catch (err) { next(err); }
}
```

Q168. How do you create a product using Express and Mongoose?

Answer: Validate the input, allowlist accepted fields, create the document, and return 201 Created. Do not accept privileged fields such as ownerId or approval status from an untrusted client unless the server explicitly sets them.

Example solution

```
export async function createProduct(req, res, next) {
  try {
    const { name, price, description } = req.body;
    if (typeof name !== "string" || !name.trim() ||
        !Number.isFinite(price) || price < 0) {
      return res.status(400).json({ message: "Invalid product data" });
    }
    const product = await Product.create({
      name: name.trim(), price, description,
      seller: req.user.id
    });
    return res.status(201).json({ data: product });
  } catch (err) { next(err); }
}
```

Q169. Show a simple authentication middleware.

Answer: Read the token from the agreed transport, verify its signature and expiry, attach the verified identity to req.user, and call next(). Handle missing/invalid tokens without exposing verification details.

Example solution

```
import jwt from "jsonwebtoken";

function requireAuth(req, res, next) {
  const token = req.cookies?.accessToken ||
    req.headers.authorization?.replace(/^Bearer\s+/i, "");
  if (!token) return res.status(401).json({ message: "Login required" });

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}
```

Q170. Show role-based authorization middleware.

Answer: Authentication identifies the caller; this middleware checks whether the verified role is allowed. Always add ownership checks for records belonging to individual users.

Example solution

```
const allowRoles = (...roles) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ message: "Login required" });
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: "Forbidden" });
  }
  next();
};

// Example:
router.post("/courses", requireAuth,
  allowRoles("mentor"), createCourse);
```

Q171. How do you hash and verify a password with bcrypt?

Answer: Hash the password before saving it. During login, compare the submitted password with the stored hash. Use an appropriate work factor and never return the hash in an API response.

Example solution

```
import bcrypt from "bcrypt";

const passwordHash = await bcrypt.hash(password, 12);
// During login:
const valid = await bcrypt.compare(password, user.passwordHash);
if (!valid) return res.status(401).json({ message: "Invalid credentials" });
```

Q172. How do you implement atomic first-come-first-served reservation?

Answer: Use a conditional atomic update in MongoDB. The filter requires the item to still be available. Under concurrent requests, only one update can change that state; the others receive no match. Add authentication and validate the item ID.

Example solution

```
const item = await Item.findOneAndUpdate(
  { _id: itemId, status: "available" },
  { $set: { status: "reserved", reservedBy: req.user.id,
            reservedAt: new Date() } },
  { new: true, runValidators: true }
);

if (!item) {
  return res.status(409).json({
    message: "Item is no longer available"
  });
}
return res.status(200).json({ data: item });
```

Q173. How do you prevent overselling inventory?

Answer: Use an atomic conditional decrement where stock is sufficient, such as filter { _id, stock: { $gte: quantity } } and update $inc: { stock: -quantity }. Validate quantity as a positive integer, and coordinate order/payment state carefully.

Q174. How would you implement search with pagination?

Answer: Normalize and validate the query, limit page size, use indexes that match the search strategy, and return page metadata. For large or frequently changing collections, consider cursor pagination instead of large skip values.

Q175. How do you prevent a user from editing another user's profile?

Answer: Use the authenticated user ID from verified credentials, not an ID supplied by the request body. Query/update with both the target ID and owner ID, or check ownership before updating. Staff access should be explicit.

Q176. How do you handle duplicate email registration?

Answer: Normalize email consistently and create a unique index on the normalized field. Catch MongoDB duplicate-key error code 11000 and return a conflict response such as 409. A pre-check can improve messaging but cannot replace the unique index.

Q177. How would you implement logout with JWT?

Answer: With cookie-based tokens, clear the cookie using matching path/domain/SameSite options. If refresh tokens are used, revoke or rotate the refresh-token record. A stateless access JWT remains valid until expiry unless the server maintains a revocation strategy.

Q178. What if an API request succeeds in the database but the client times out?

Answer: The client may not know whether the operation completed. For non-idempotent operations, use an idempotency key stored with the operation result, or design the endpoint so retries are safe. Do not blindly submit a second payment/order.

Q179. How would you handle a payment webhook?

Answer: Verify the provider's signature against the raw request body, reject invalid signatures, store the provider event ID with a unique constraint, and process idempotently. Acknowledge promptly and move slow work to a queue. Never trust a frontend redirect as proof of payment.

Q180. How would you implement an order status lifecycle?

Answer: Define allowed transitions, such as pending → paid → packed → shipped → delivered, with cancellation rules. Enforce transitions on the server, record timestamps and actors, and prevent arbitrary client-supplied status changes.

Q181. How do you handle stale data in a React component?

Answer: Cancel the previous fetch with AbortController or ignore its result in cleanup. This prevents a slower old request from overwriting a newer search result. For complex data fetching, use a library that manages caching and request lifecycles.

Q182. How do you debug CORS errors?

Answer: Check the exact frontend origin, scheme, host, and port; inspect the OPTIONS preflight in the Network tab; ensure allowed methods/headers are correct; and configure credentials consistently. Do not use wildcard origin with credentialed cookies.

Q183. What would you do if the deployed frontend works but API calls fail?

Answer: Inspect the browser Network tab and console, verify the production API base URL, server health/logs, CORS allowlist, HTTPS/mixed-content rules, environment variables, database connectivity, and route prefixes. Reproduce the exact failing request outside the browser too.

Q184. How would you improve a slow MongoDB endpoint?

Answer: Measure query duration and inspect explain plans. Add a suitable index, project only required fields, paginate, avoid unbounded populate, reduce repeated queries, and cache only if the data and invalidation rules allow it.

Q185. How do you design a cart in MongoDB?

Answer: A cart can reference a user and contain product IDs, quantities, and price snapshots as appropriate. Validate stock and current prices again at checkout; do not trust client-calculated totals. Consider atomic stock reservation/decrement during order creation.

Q186. What is the difference between storing a product price and a price snapshot in an order?

Answer: The product's current price can change. An order should preserve the price agreed at purchase time, so store a server-calculated line-item price snapshot and currency in the order. Never trust a submitted total from the browser.

Q187. How would you explain your MERN project in an interview?

Answer: Use a concise structure: problem and users; architecture; your individual contribution; key features; data model and API design; one technical challenge and how you solved it; security/testing/deployment; and what you would improve. Be precise about what you personally implemented.

Q188. How do you explain a technical trade-off?

Answer: State the requirement, options considered, decision criteria, chosen approach, downside, and when you would choose differently. Example: a modular Express monolith is simpler to deploy for a student project than microservices, while preserving clear module boundaries.

Q189. What should you do when you do not know an interview answer?

Answer: Be honest. Explain what you do know, reason from first principles, state assumptions, and describe how you would verify the answer. Avoid bluffing; interviewers often value a clear problem-solving process.

Final interview checklist

■ Can I explain the complete request flow from React to Express to MongoDB and back?

■ Can I explain useState, useEffect, useRef, useMemo, useCallback, and custom hooks with examples?

■ Can I build and test CRUD REST endpoints without relying on the frontend?

■ Can I explain JWT verification, password hashing, role checks, and resource ownership?

■ Can I explain why frontend validation is not enough and how MongoDB unique indexes prevent races?

■ Can I debug CORS, environment-variable, deployment, and database-connection issues?

■ Can I explain one difficult bug, one design trade-off, and my exact contribution to each project?

Suggested revision order

JavaScript fundamentals → HTTP/REST → React → Node/Express → MongoDB/Mongoose → Authentication & security → Project scenarios. After each section, close the PDF and try answering aloud before checking the explanation.